

The Pre-Meeting Knowledge System

How Proxy learns a company's software before it joins a meeting. Part A is the product — what it does and every capability it unlocks. Parts B–C are the build — the architecture and, tool by tool, exactly what each piece is, how we wire it in, and how we optimize it, specific enough to implement.

Before Proxy can catch a wrong number or answer "can our system even do this?", it has to genuinely *know* the company's software. This system builds that knowledge and keeps it true. It runs entirely before any meeting.

THE ONE IDEA TO HOLD

We build a **living map of the company's software** — a real graph of dots (systems, files, concepts) joined by labeled lines (which calls which, which shares a database with which) — then cut that map into team-sized pieces and train **one AI expert per piece** until it knows that piece cold. Everything below is how we draw the map accurately, cut it well, train the experts, and stay honest about what we haven't mapped yet.

PART A — THE PRODUCT

What it does, and everything it makes possible

The capabilities a product developer needs to know — written before any implementation detail.

This system produces three things, and every later part of Proxy reads from them.

- **The Estate Graph** — the map. One plain-English "page" of understanding for every real thing in the company (a service like `checkout`, a team, a key concept), plus a fine-grained graph of the actual code (files, functions, and how they call each other). Every fact on every page is backed by a citation to the real source, or it does not appear.
- **Per-unit expert agents** — the map is cut into team-sized territories, and we train one AI agent per territory until it is a genuine authority on it. When a meeting touches `checkout`, the `checkout` expert shows up already knowing it — its code, its owners, and everything it connects to.
- **The coverage ledger** — an honest, running list of what we *don't* know yet, each item tagged with the specific person who probably does. This is what lets Proxy say "I'm not certain — Maya would know" instead of guessing.

The capabilities this unlocks

Because the map is a real graph of typed relationships, Proxy can do things a summarizer never could. It can answer "**what breaks if we change checkout?**" by walking every line out of the `checkout` dot. It can answer "**who owns this?**" instantly. It can answer "**is that true?**" by going to the exact cited file. And critically, it knows the edge of its own knowledge — so it can be trusted to act, because it abstains rather than bluff.

It works for any company, at any size

The map is **fractal**: the same shape at every scale. A two-person startup is one dense cluster of dots. A ten-thousand-person enterprise is that same shape repeated — each team is its own cluster, and the whole organization is a smaller graph of team-clusters joined by a few important lines (which team's service calls which). We never try to hold "the whole company" in one place; we hold one team-sized piece and its connections. That is why the identical machinery runs on a solo founder's single repository and on one product-area of a giant enterprise — nothing in it assumes a size.

It works for any kind of company

Nothing in this system knows or cares what industry the company is in. It reads code, configuration, and documents and builds a graph of whatever is there — a fintech's payment services, a healthcare company's data pipelines, a game studio's engine modules. The tools that parse code are language-based, not industry-based. The experts are trained on *whatever the estate contains*. There is no industry-specific logic anywhere, so a new vertical needs zero new code.

It takes in more than code

Software is never only code. A "manual door" lets anyone drag in or paste a document, a diagram, a spreadsheet, or a note; it flows through the same understanding and permission process as code. Office files, PDFs, and slides are all handled. This is also the zero-setup on-ramp — a team can get value before connecting a single repository, just by handing Proxy the design doc for today's meeting.

PART B — HOW IT WORKS

The architecture, end to end

The whole flow in one picture, then in words.



Figure 1 — The pre-meeting pipeline. The first four stages build the map; training turns it into experts; maintenance keeps it true and learns from every meeting.

The flow reads left to right. **Connect** gets read-only access to a team's code and documents.

Skeleton uses fast parsers (no AI, pennies) to turn raw code into the structural bones of the graph — which file calls which, which imports which. **Pages** sends AI reader-agents across that skeleton to write the plain-English, cited understanding on top. **Carve** cuts the finished graph into team-sized units small enough for one AI to master. **Train** gives each unit its own expert agent and puts it through a real study-and-exam process. **Maintain** runs forever: re-reading only what changed, and folding every meeting's decisions back into the map so it keeps getting smarter.

Two kinds of worker appear here, and the difference matters for cost. The **mechanical extractors** (Skeleton) are ordinary programs — no AI, so they are effectively free and can run constantly. The **reader and expert agents** (Pages, Train) are AI, so we run them carefully, in the background, and cache their results.

What "a graph" actually means here

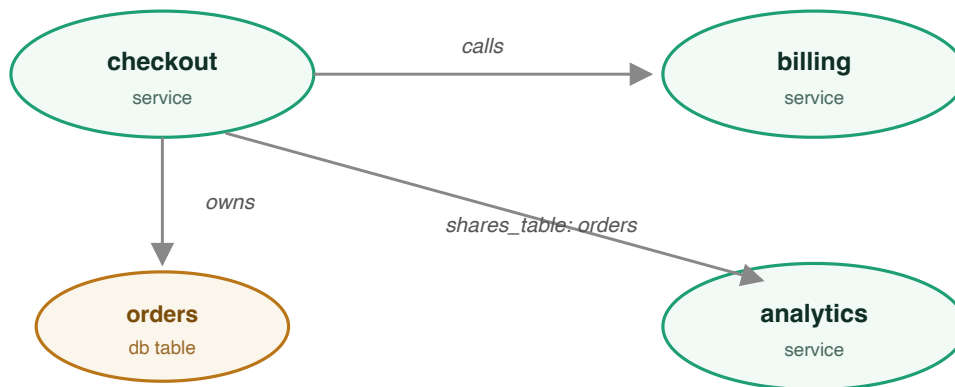


Figure 2 — A real slice of the Estate Graph. Dots are nodes; labeled arrows are typed edges. "shares_table" — two services on one database table — is the strongest and most dangerous hidden coupling, and it is exactly the kind of thing a wrong decision in a meeting misses.

A graph is just **nodes** (the dots) and **edges** (the labeled lines). We never say two things are vaguely "connected" — every edge has a **type**: `calls`, `imports`, `shares_table`, `api_contract`, `discusses`, `owns`. That typing is what makes the graph answer real questions.

THE GRAPH IS REAL; THE DATABASE IS BORING ON PURPOSE

We store the graph as two tables in **Postgres** — one for nodes, one for edges — because Postgres is cheap, proven, and something every engineer already knows. The graph is completely real; Postgres is just the filing cabinet. We walk it as a graph using recursive queries (follow edges outward from a node), and if we ever need heavier graph operations we switch on **Apache AGE**, an extension that adds a real graph query language *inside* the same Postgres — so we never run or pay for a second database.

PART C — THE BUILD

Every piece, exactly — what it is, how we wire it, how we optimize it

For each tool: first the overview (what it is and why it's here, in plain terms), then the technical specifics (how we integrate and tune it). Dense, but built to be understood.

Stage 1 — Connect: getting access, safely and minimally

Overview. A company connects one team's code repositories and document spaces. We ask for the *least possible* access — read-only code and configuration, never production systems, customer data, or live monitoring. This single choice is what lets a team say yes in an afternoon instead of after a three-month security review, and it's a promise we can state plainly.

Nango — the key-holder

What it is and what we do with it. Nango is an open-source service that manages "OAuth tokens" — the digital keys that let one app read another (our app reading the company's GitHub). We self-host it (it's free), so those keys live entirely inside our infrastructure. Nango's only job is to hold the keys, keep them fresh, and pass our read requests through.

How we wire and optimize it. We run Nango as a service beside our database. Every read (a repo pull, a Jira fetch) goes through it, so credentials are never scattered across our code. The same key-holder is reused later by the action layer: when Proxy needs to *do* something in a meeting, a gateway mints a short-lived, narrowly-scoped token from Nango's stored key, so the sandbox doing the work never holds a real secret. One integration, used in two places.

Permission stamps — how one shared map stays private

Overview. Everything we ingest is stamped with *who is allowed to see it*, copied directly from the source system's own permissions. This is what makes a single shared graph safe for every team: a person only ever sees what they could already open themselves.

Technical specifics. The stamp is a row: `permission(node_id, allowed_principals[], source_system, synced_at)`, where `allowed_principals` is the list of user or group IDs the source system says can see this file. Every knowledge query is filtered with a list-overlap check — in plain SQL, `WHERE current_user_principals && allowed_principals` (the `&&` means "do these two lists share any member"). We re-pull permissions on a change-driven cadence via webhooks, so the

moment someone loses access in GitHub their match disappears and the next query returns nothing. **Revocation is enforced at read time, never cached** — the correctness point that makes enterprises comfortable.

Stage 2 — Skeleton: turning raw code into a graph, mechanically

Overview. When code arrives, we build the structural bones of the graph with fast parsing tools — no AI, so it costs pennies and can re-run constantly. Four tools each do one job. This is the stage people usually gloss over, so here is exactly what each is and how it plugs into *our* system.

tree-sitter — read each file into a precise structure

What it is. A parser that reads a source file and turns it into a *syntax tree* — a precise, machine-readable outline of "here is a function, here's what's inside it, here's a call to another function." It's fast, incremental, and covers ~40 languages.

How we use and optimize it. We run tree-sitter *queries* (small patterns) against each file's tree to pull out declarations (functions, classes) and call sites. Because it's incremental, when one file changes we re-parse only that file in milliseconds, not the whole repo — this is what makes staying-current cheap.

CodeGraphContext — assemble the whole repo into a code graph

What it is. An open-source tool that runs tree-sitter across an *entire* repository and assembles the results into a code graph: nodes for files, functions, and classes, and edges for "this function calls that one" and "this file imports that one." On its own it can store that graph in a graph database (Neo4j) and expose it for querying.

How it involves our system — and why we don't just adopt it blindly. CodeGraphContext produces a graph; we consume that graph and pour it into *our* Postgres node and edge tables. We do this through a single seam we control — the `GraphExtractor` interface, one function:

`extract(repo) → stream of Nodes and Edges`. CodeGraphContext is one implementation behind that seam. This is the optimization that makes the whole system scale to any codebase: if a customer's language is handled poorly, or a better tool ships next year, we swap the implementation behind the interface and **nothing downstream changes** — not the storage, not the carving, not the experts. The estate graph never depends on any one vendor.

SCIP — make the edges exact, not guesses

What it is. A precise code-indexing standard (the technology behind Sourcegraph's "go to definition"). Language indexers like `scip-python` and `scip-typescript` emit an index of every

symbol's exact definition and every place it's used, with file-and-line precision.

How we use it. tree-sitter can see "there's a call to something named `charge`," but not *which* `charge` across a big codebase. We load the SCIP index to **resolve** that — upgrading a guessed edge ("calls something called charge") into a resolved, cited edge ("calls `billing/client.py:charge`"). Every edge in the graph carries that citation, which is what lets the whole product cite its sources.

Zoekt — instant search, used live in meetings

What it is. A blazing-fast code search index (Google's, open-sourced) built on trigrams, so any regular-expression search over millions of lines returns in milliseconds.

How we use and optimize it. Zoekt is not just for building — it's queried *live during meetings*. When an expert needs to confirm the current behavior of a line, it searches Zoekt rather than trusting its trained memory, so it always cites the code as it is *right now*. We keep the index warm for connected repos, so a live lookup is a few milliseconds.

WHAT STATIC PARSING CAN'T SEE, AND HOW WE RECOVER IT

Some connections are decided while the program runs — code that picks which service to call from a configuration value. Those don't appear in the syntax tree. We recover them two honest ways: (1) parse the **config and infrastructure files**, which literally list the real endpoints and database names, and (2) run the code on **fake, synthetic inputs inside a sandbox** and watch what it reaches. We never touch a live system to learn this. Whatever is still unresolved becomes a known-unknown (Stage 6) — flagged, never guessed.

Stage 3 — Pages: AI reads the skeleton and writes the meaning

Overview. The skeleton knows *structure* ("A calls B") but not *meaning* ("checkout captures payment and must never double-charge"). Now AI reader-agents read the code and write one plain-English, cited page per real thing.

What "an AI agent" concretely is here. Each reader is a session of the **Claude Agent SDK** — a loop that runs: think → call a tool → read the result → repeat, until the job is done. We give it a fixed set of tools (each a function it may call) and a system prompt that defines its job and its hard rules. The important rule is enforced by a tool, not by hope: the `write_page` tool *rejects* any sentence lacking a citation, so an uncited claim physically cannot land on a page.

SYSTEM PROMPT — READER AGENT (ILLUSTRATIVE)

You are the reader for the unit `{unit_name}`. Write one factual page per real thing in this unit (a service, module, or key concept).

Hard rules:

- Every sentence must cite a real file:line or document id. Uncited sentences are rejected by `write_page`. Never write what you cannot cite.
- Prefer reading the source over guessing. If a reference leads nowhere, or you hold a belief you cannot cite, call `note_unknown()` instead of writing it.

Tools: `read_file(path)`, `search_code(query)` # Zoekt,
`find_refs(symbol)` # SCIP, `run_in_sandbox(code)` # E2B,
`write_page(node_id, body, citations[])`, `note_unknown(what, why, likely_owner)`

Read the files in the given order (dependencies first). Keep a running memory of what you have learned; each page should build on it.

How we optimize it. All of this reading is offline work with no deadline, so we run it on the **Batch API** — the same models at roughly half price, in exchange for asynchronous turnaround. Documents are converted first (Office files by **MarkItDown**, PDFs by **Docling**) and linked to the code they describe. Tickets are read *live at meeting time*, never stored, because they go stale within hours.

Stage 4 — Carve: cut the map into pieces one AI can master

Overview. A whole company is far too much for one AI to know well. So we cut the graph into **units** — team-sized territories of roughly 10–40 files, small enough that everything about a unit fits in one AI's working memory (about 200,000 words of context). The cutter finds the *natural* clusters: the groups of files that talk to each other far more than to anything outside.

Leiden community detection — exactly how we run it

What it is. Leiden is a well-established algorithm for finding communities (tightly-connected clusters) in a graph. We use the `leidenalg` library. But feeding it the raw code graph gives mediocre cuts; the craft is in how we prepare and tune it. In order:

- **Build a weighted graph** (in the `igraph` library): one dot per file, plus document dots. Each edge gets a weight = a base amount for its *type* (`shares_table` heaviest, `ticket_link` lightest) *plus* a "meaning" weight — how similar the two files' content is, measured by comparing their embeddings (numeric fingerprints of meaning). So "coupled" means both *structurally* connected and *about the same thing*.
- **Excise the super-hubs first.** Shared utilities (logging, auth helpers) are connected to *everything*, so if left in they smear every cluster into mush. Before clustering, we compute each dot's

connection count and lift the top $\sim 1\text{--}2\%$ out into a separate pinned `platform` layer. *This one step is the difference between clean units and useless ones.*

- **Run Leiden with a resolution knob (γ).** γ controls cluster size — higher γ makes smaller clusters. We tune γ so the *median* unit lands in the 10–40-file band. The multi-level form of the algorithm gives us nested levels for free: file $\rightarrow$ unit $\rightarrow$ domain $\rightarrow$ whole org.
- **Enforce the size band.** A unit that comes out too big is re-run through Leiden at a higher γ to split it; fragments too small are merged into their strongest-connected neighbor. Ownership is assigned by majority vote of the CODEOWNERS file over the unit's files.
- **Pin stable IDs.** On every re-scan we match new clusters to old ones by how much they overlap (shared files $\div$ total files); above $\sim 50\%$ overlap the unit keeps its old ID. This is why last week's `checkout` unit is still `checkout` after today's code changes — the map never silently reshuffles under the experts trained on it.

The product view. A human owner sees the proposed cut once — "we grouped these 22 files as *checkout*, these 15 as *billing*" — and can rename, merge, or split in a couple of minutes. After that it's automatic and stable.

Stage 5 — Train: turn each piece into a genuine expert

Overview. For each unit we train one agent until it's an authority — not "load the files into a prompt," but a real study-and-exam process, run cheaply in the background. Here is precisely how each part works, including the parts that are easy to hand-wave.

Read in dependency order — so the reading actually makes sense

From the unit's internal edges we build a dependency graph and **topologically sort** it (a standard ordering where nothing comes before the things it depends on — computed with Kahn's algorithm). We read files in that order: leaves first (files that depend on nothing), then files that depend only on already-read files. Why it matters: when the agent reads `pay.py`, everything `pay.py` relies on is *already understood and summarized in its memory*, so the new file makes sense instead of being full of unknown references. The agent also reads one hop *past* the unit's boundary, so it learns its **blast-radius** — who it calls, who calls it, which tables it shares.

The self-quiz — the actual questions, and how we grade them

After writing its pages, the expert tests itself. It generates 40–80 questions that *target each load-bearing claim it wrote*, in three kinds: **relational** ("what calls `capture_payment` ?"; "who writes to the `orders` table?"), **factual** ("what does `capture_payment` return when billing times out?"), and **behavioral** ("what happens on a duplicate charge?"). Then — the crucial part — it grades every

answer *against reality, never against its own confidence*: relational questions are checked by *running the actual Zoekt/SCIP query* and comparing the true answer to the agent's; factual and behavioral ones by re-reading the exact cited lines or *running the code in the sandbox* and observing. A mismatch **deletes the wrong page claim** and triggers a re-read.

SYSTEM PROMPT — EXPERT TRAINING (ILLUSTRATIVE)

You are becoming the authority on the unit `{unit_name}`.

Phase 1 — Study. Read the files in the given dependency order. Maintain a memory of cited facts. Write the unit's pages (every sentence cited).

Phase 2 — Self-test. Generate 40–80 questions about YOUR territory:

- relational (who calls X? who writes table Y?)
- factual (what does X return when Z?)
- behavioral (what happens when W?)

For EACH: answer it, then VERIFY the answer against live source —

run the Zoekt/SCIP query, re-read the cited lines, or run it in the sandbox.

If your answer disagrees with the source: delete that page claim, re-read, rewrite. Never trust your confidence; only a verified check counts.

Phase 3 — Record gaps. Anything you cannot verify → `note_unknown(what, why, likely_owner)`. Do not paper over it.

The final exam — graded on the team's own history

Trivia isn't understanding, so the real exam is built from the team's *own* closed issues and merged pull requests. Each becomes a question with a *known* answer (the PR's own description and diff): "PR #451 fixed a payment bug — what was the root cause?" The expert answers blind, and its score is the fraction it gets right. It must clear a floor (about 80%) before it is ever allowed to speak in a meeting. This tests real understanding of the team's actual problems.

Freeze and cache

The finished "pack" — the pages, the memory, the tool manifest, the owner map — is **prompt-cached with a one-hour time-to-live**, so waking this expert in a meeting is a cache hit: instant and cheap. Prompt caching means the model keeps the expert's context ready-to-go rather than re-reading it from scratch each time.

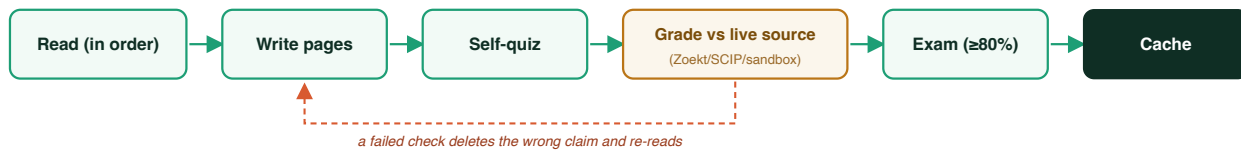


Figure 3 — The training loop. The orange feedback path is the point: the expert is graded against the live source, and a wrong answer forces a re-read, so it earns its knowledge rather than asserting it.

Stage 6 — The coverage loop: what happens when we don't know

Overview. Every knowledge system is judged on how it handles ignorance. Ours is built to *never guess silently*. This is what makes Proxy trustworthy enough to act.

How it works. Five conditions raise an "I don't know" flag: a reference that leads nowhere, an answer that failed its grade, a document that contradicts the code, two neighbor experts that disagree on a shared boundary, or a belief that can't be cited. On any flag, the expert runs a fixed ladder — **self-resolve first** (widen the search, read one hop further out, run the code in the sandbox), and only if still stuck, write a typed record and ask the right human.

Know-who-to-ask is a real feature, not a fallback. The record is `unknown(unit_id, description, why_matters, likely_owner, resolver, re_verify_at)`. `likely_owner` is computed during training as the overlap of three signals: the CODEOWNERS file, git blame history, and who resolves related tickets. So Proxy always knows the *specific* person to ask, and it asks a specific question, batched into one Slack message with a one-click answer: "Maya — you last touched the retry logic; does it retry on a timeout?" The answer writes back onto the page with an owner and a re-check date. Until then, the expert simply **abstains** on that point in meetings. It would rather say nothing than say something wrong — and that discipline is the entire reason a company lets it act.

Stage 7 — Maintain: stay current, and learn from meetings

Overview. Code changes constantly and meetings produce knowledge that never lived in any file. Maintenance handles both, cheaply.

Staying fresh. Webhooks (a GitHub push, a merged PR) trigger a re-scan of *only the changed files*; any page whose source touched a changed file is marked stale and re-verified. It's a diff, not a rebuild. Right before a meeting, we re-verify just that delta for the relevant experts and warm their caches, so they walk in current. (In the first version, since storage is nearly free, we simply warm *all* the experts and let Proxy join with a bit of manually-entered context about the meeting.)

Learning from meetings. When a meeting decides something ("we're deprecating the old billing path"), that decision is written onto the affected pages as a new cited fact, with the meeting as its source. If it conflicts with what we believed, we don't overwrite — we keep both, mark the new one with a timestamp, and re-verify (a database `UPSERT` with an `ON CONFLICT` rule that supersedes-with-timestamp). The company's acronyms and unwritten "who calls what what" accumulate into a living glossary. Every meeting makes the next one smarter — the map compounds human insight on top of technical insight.

How the whole system stays cheap, fast, and size-proof

- **Cost.** The skeleton is compute-only (no AI). The only real spend is the one-time expert training, run on the Batch API (~half price) and prompt-cached — landing around low hundreds of dollars for a design partner's first month. Re-checks are delta-only fractions of that.
- **Freshness at scale.** Every node carries a freshness timestamp; incremental parsing and delta re-index mean we only ever touch what changed, so a million-file monorepo costs the same to keep current as a small one — proportional to *change*, not size.
- **The adapter is the scaling lever.** Because every extractor sits behind the one `GraphExtractor` interface, and every document parser behind a format adapter, we onboard a new language, a new source, or a better tool by writing one adapter — never by touching the store, the carve, or the experts.
- **Size-proof by shape.** The fractal graph means we never process "the whole company." One team-sized unit and its edges is the largest thing we ever hold at once, so the same pipeline runs identically at every scale.

Done when

This system is complete when: every page claim carries a live-source citation and nothing uncited survives; the permission filter provably works, including immediately after someone's access is revoked; the carve yields ~10–40-file units with stable IDs that don't reshuffle on re-scan; each expert clears its closed-ticket exam floor before it is allowed into a meeting; on any realistically messy codebase the unknowns list is non-empty and every item names an owner and a resolver; a pre-meeting refresh re-checks only the delta and finishes before the meeting starts; a decision made in a meeting appears as a cited fact on the right page afterward; and the identical pipeline runs on a one-repo startup and one product-area of a large enterprise, with nothing industry-specific anywhere in it.